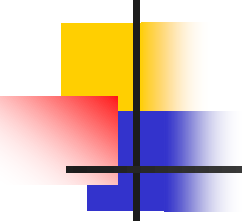


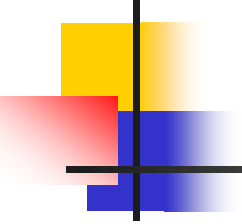
Chapter 9

Graph



OBJECTIVE

- Introduces:
 - ✓ Graph terminology and concept
 - ✓ Graph storage structures
 - ✓ Traversing operation
 - ✓ Determining shortest path



CONTENTS

9.1 Introductions

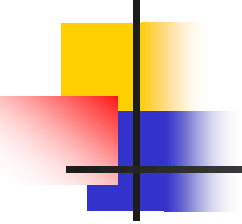
9.2 Graph storage structures

9.3 Graph Traversing

9.4 Networks

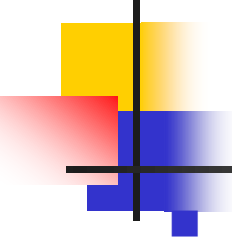
9.4.1 Shortest Distance

9.4.2 Spanning Tree



9.1 Introduction

- A graph is a collection of nodes, called **vertices**, and line segments, called **arcs** or **edges**, that connect pairs of nodes.
- The graph is used for modeling any information used in computer applications.
- Examples :
 - i) Represents relationship amongst cities – where the nodes represent cities and line segments are distances.
 - ii) The World Wide Web. The files are the vertices. A link from one file to another is a directed edge (or *arc*).



In general, the graph has always been used because it represents cycle and also has more than one connection.

- Graphs maybe indirected or directed:
 - ✓ *Directed Graph* (or digraph) –each line called an **arc**, has a direction indicating how it may be traversed. Figure 1(a) shows the example of directed graph.
 - ✓ *Undirected graph* –there is no direction on the lines known as **edges** and it may be traversed in either direction. Figure 1(b) shows the example of undirected graph.

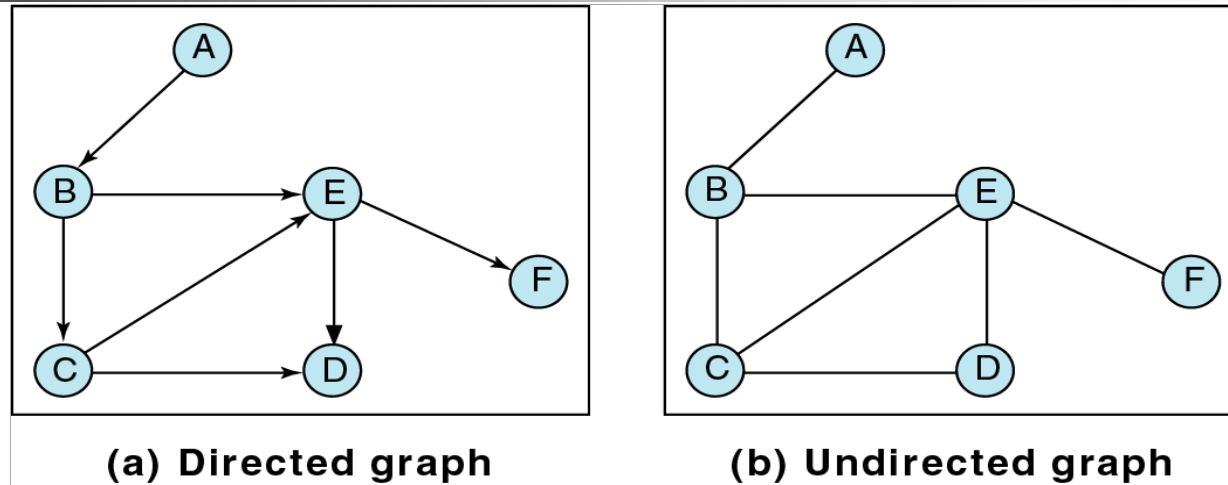
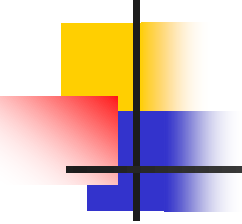
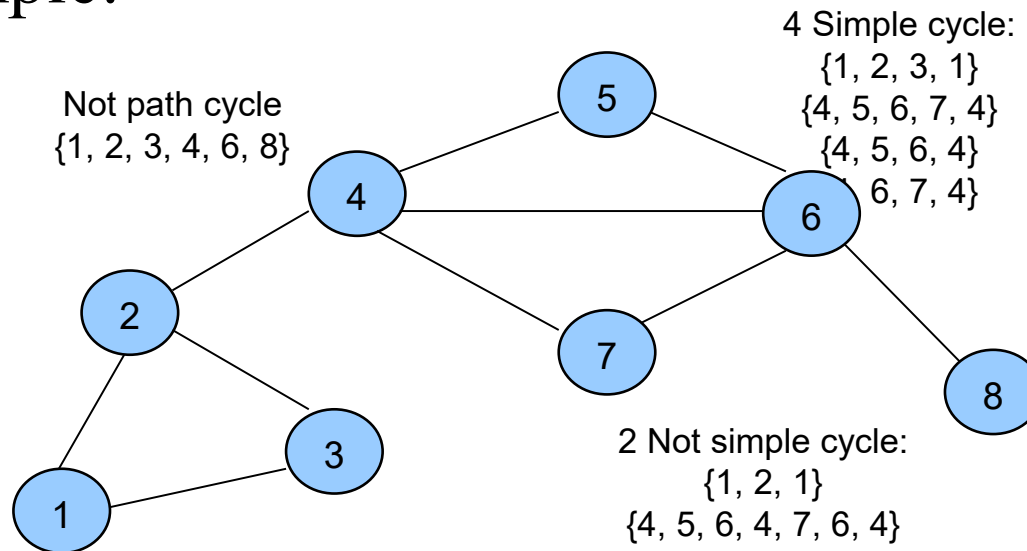


Figure 1: Graphs

- Two vertices in a graph are said to be adjacent vertices (or neighbors) if an edge directly connects them. In Figure 1, A and B are adjacent, whereas D and F are not.

- 
-
- A cycle is a path consisting of a least three vertices that starts and ends with the same vertex. (Example: In Figure 1(b) {B, C, D, E, B} is a cycle.
 - A graph $G=(V,E)$ means that the graph consists a set of vertices (V) and edges (E) which connect them.

- Example:



- A graph is said to be connected if there is a path from any vertex to any vertex.

- A graph is disjoint if it is not connected. In other words, a node that has no connection with another nodes. Figure 2(c) shows one of the examples of disjoint graphs.

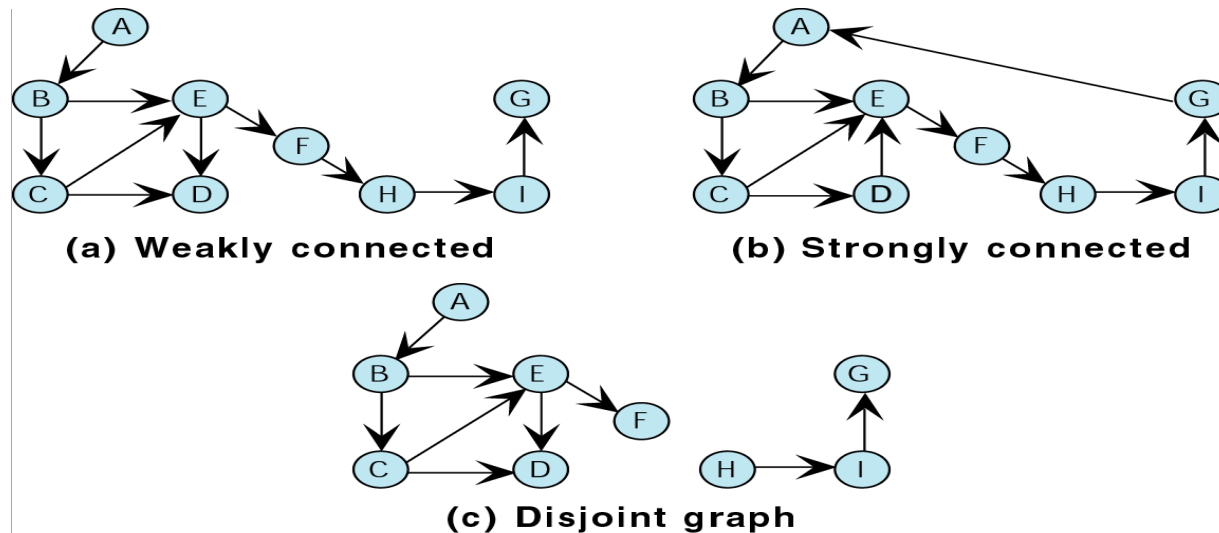
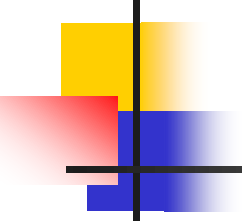
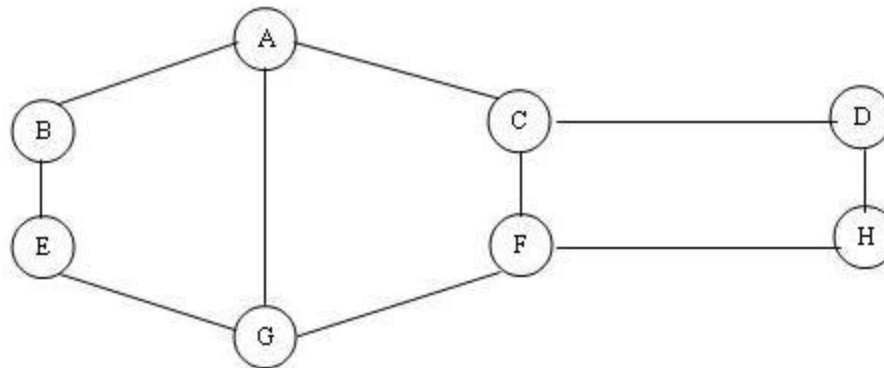


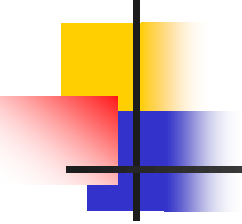
Figure 2 Connected and disjoint graphs

- 
-
- The **degree** of a vertex is the number of lines incident to it.
 - The **indegree** is the number of arcs entering the vertex.
 - The **outdegree** of a vertex in a graph is the number of arcs leaving the vertex. (Example: Figure 2(a), the indegree of vertex B is 1 and its outdegree is 2).

- Exercise:

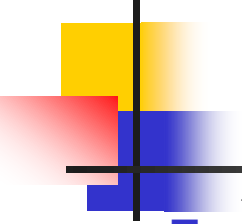


- Find the degree, outdegree, and indegree of vertices A, E, F, G, and H.



9.2 Graph Storage Structures

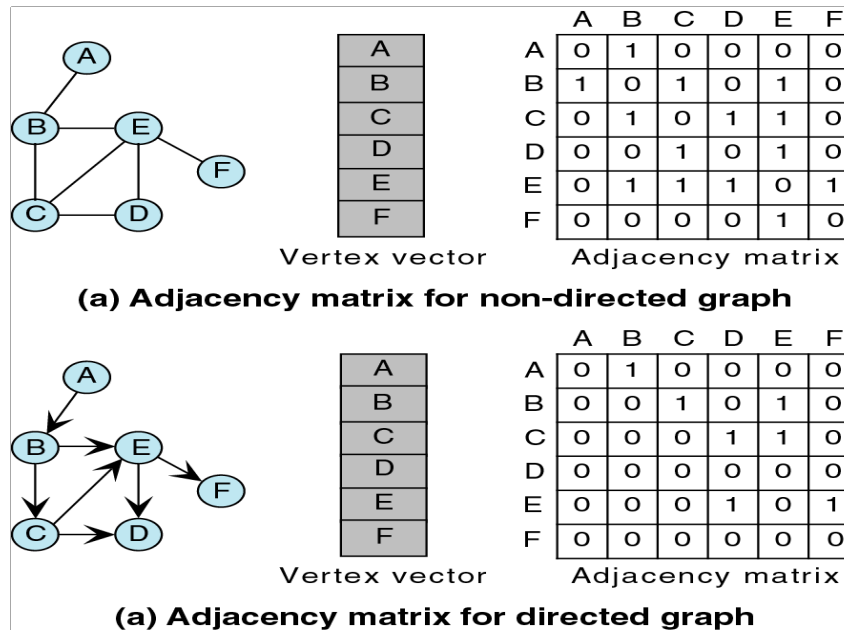
- To represent a graph we need to store two sets to represent the vertices of the graph and the edges or arcs.
- Two most common structures used to store the sets are arrays and linked lists.

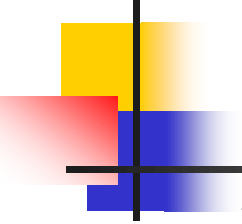


9.2.1 Adjacency Matrix

- For a graph with n nodes (1, 2, 3, 4, ..., n), the adjacency matrix is the $n \times n$ matrix, in which, if there is an edge between vertices, the entry in row i and column j is 1 (true) and is 0 (or false) otherwise.
- The adjacency matrix for non-directed graph is symmetry, $A_{ij} = A_{ji}$
- The adjacency matrix for directed graph is not symmetry, $A_{ij} \neq A_{ji}$

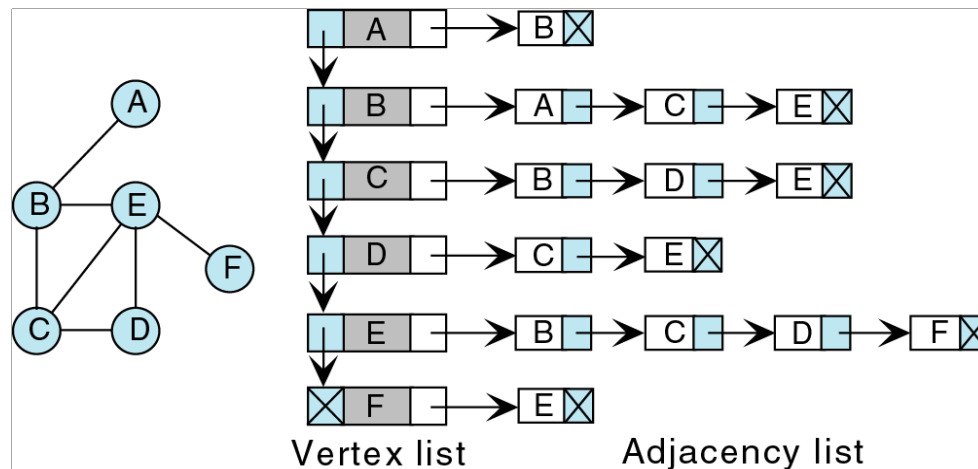
■ Example:

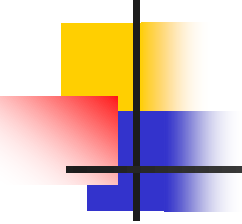


- 
-
- Limitations in the adjacency matrix:
 - ✓ The size of the graph must be known before the program starts
 - ✓ Only one edge can be stored between any two vertices

9.2.2 Adjacency List

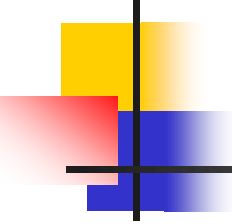
- Use a linked list to store the vertices. The pointer at the left of the list linked the vertex entries. In this method, the vertex list is a singly linked list of the vertices in the list. An adjacency list is shown in the figure below:
- Example:

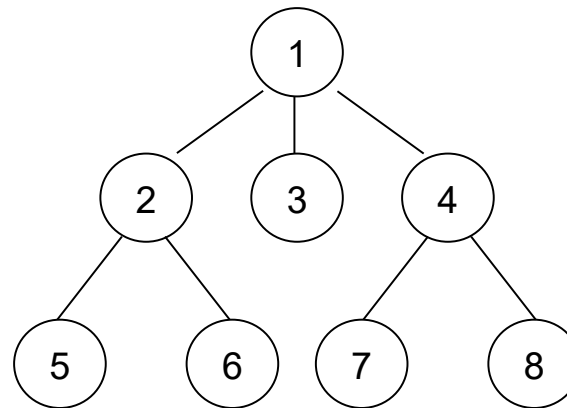




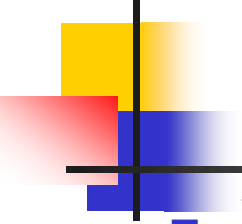
9.3 Traversing Graph

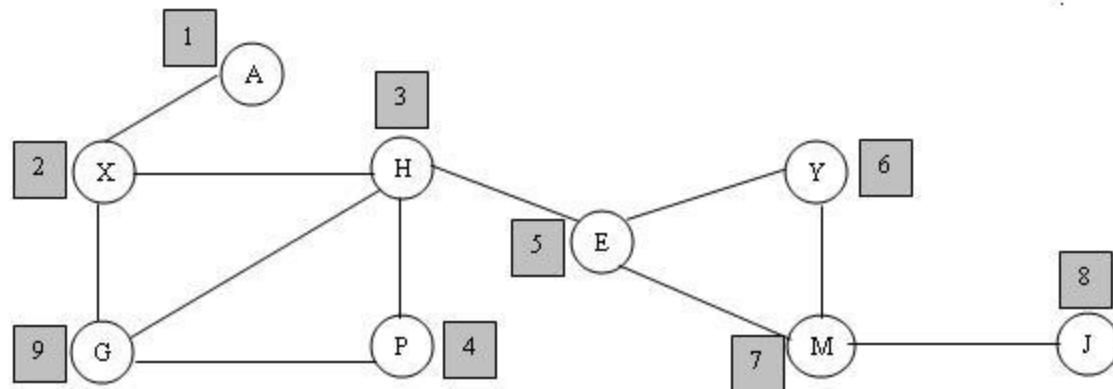
- Two standard graph traversals are depth first and breadth first.
 - ✓ Depth-first
 - In the depth-first traversal, all of node's descendents are processed before moving to an adjacent node.
 - ✓ Breadth-first
 - In the breadth-first traversal all adjacent vertices are processed before processing the descendents of vertex.

- 
- Example: Depth-first traversal and breadth-first traversal of a tree



- ✓ Depth-first traversal: 1 2 5 6 3 4 7 8
- ✓ Breadth-first traversal: 1 2 3 4 5 6 7 8

- 
- Example: Depth-first traversal and breadth-first traversal of an undirected graph
 - Depth-first traversal of an undirected graph starts by processing the first vertex of the graph. After processing the first vertex, select any vertex adjacent to the first vertex and process it. As we process each vertex, we select an adjacent vertex until we reach a vertex with no adjacent entries (similar to reaching a leaf in a tree). We then back out of the structure, processing adjacent vertices as we go. A stack (or recursion) is required.

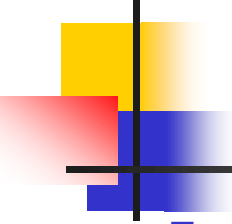


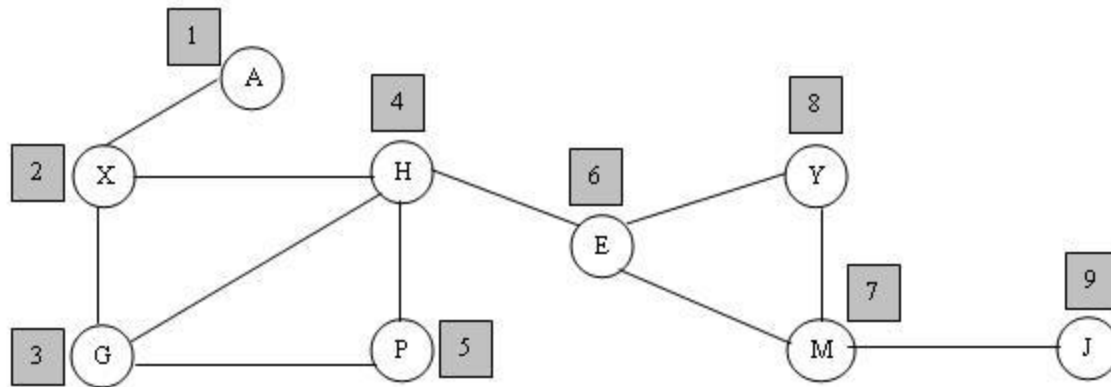
- ✓ Depth-first traversal: A X H P E Y M J G
- ✓ Stack contents:

				P		Y				
A			H	E	E	M	M	J		
	X	G	G	G	G	G	G	G		G
1	2	3	4	5	6	7	8	9		

- Algorithm:

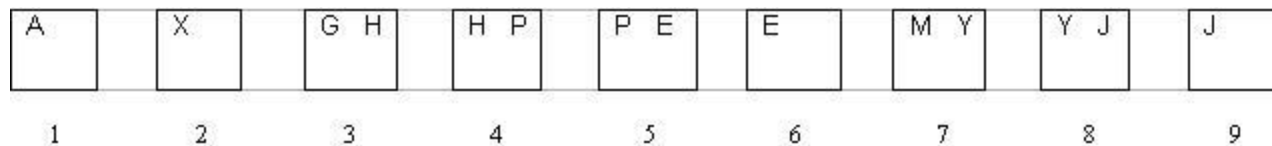
- 1) Push the first vertex, A, into the stack.
- 2) Loop, pop the stack, and after processing the vertex, push all of the adjacent vertices into the stack, process it, and then push G and H into the stack, shown as Step 3 in the above diagram.
- 3) When the stack is empty, the traversal is complete.

- 
-
- In the breadth-first traversal of an undirected graph, we process all adjacent vertices of a vertex before going to the next level. Begin by picking a starting vertex; after processing it, we process all of its adjacent vertices. After we process all of the first vertex's adjacent vertices, we pick the first adjacent vertex and process all of its vertices, then the second adjacent vertex and process all of its vertices, and so forth until we are finished.



✓ Breadth-first traversal: A X G H P E M Y J

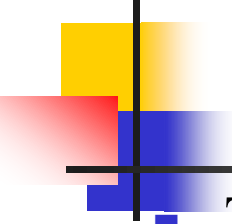
✓ Queue contents:



■ Algorithm:

- 1) Enqueue vertex A in the queue.
- 2) Loop, dequeue the queue and process the vertex from the front of the queue. After processing the vertex, place all of its adjacent vertices into the queue. Therefore at Step 2, dequeue vertex X, process it, and then place vertices G and H in the queue. In Step 3, process vertex G. When the queue is empty, the traversal is complete.

■ Example: Depth-first traversal and breadth-first traversal of a directed graph.



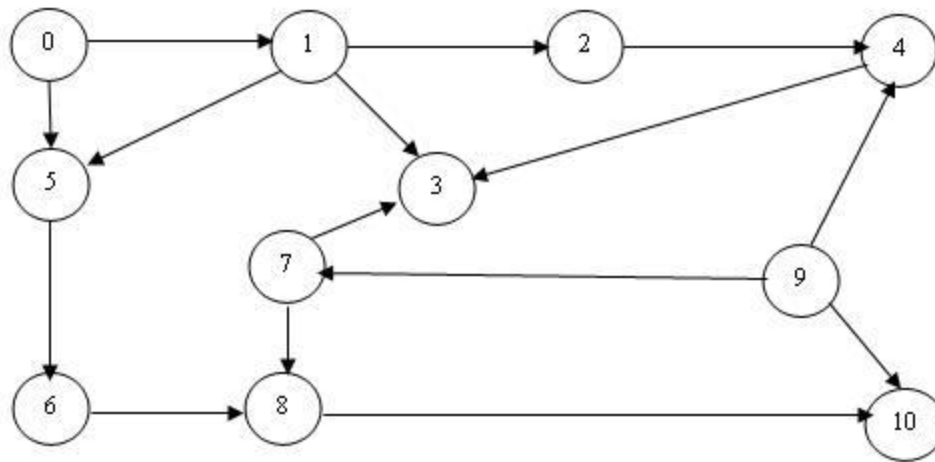
- The depth-first traversal of a directed graph is similar to the preorder traversal of a binary tree. The general algorithm is:

for each vertex, v , in the graph

if v is not visited

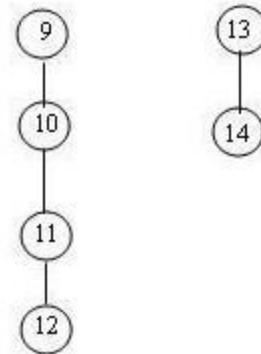
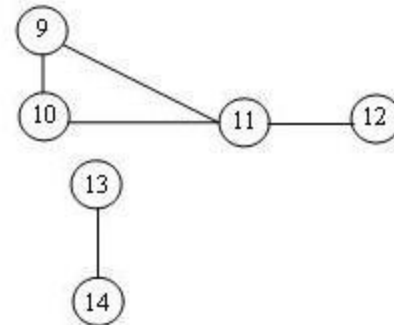
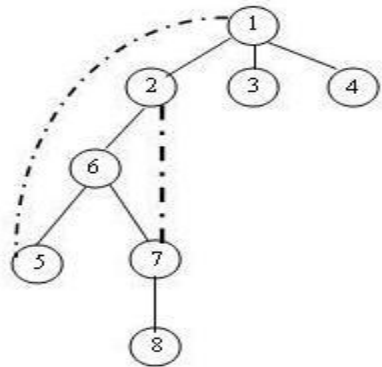
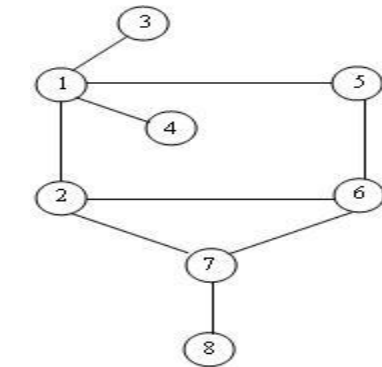
start the depth first traversal at v

- The breadth-first traversal of a directed graph is similar to traversing a binary tree level by level (the nodes at each level are visited from left to right). All the nodes at any level, i , are visited before visiting the nodes at level $i + 1$.



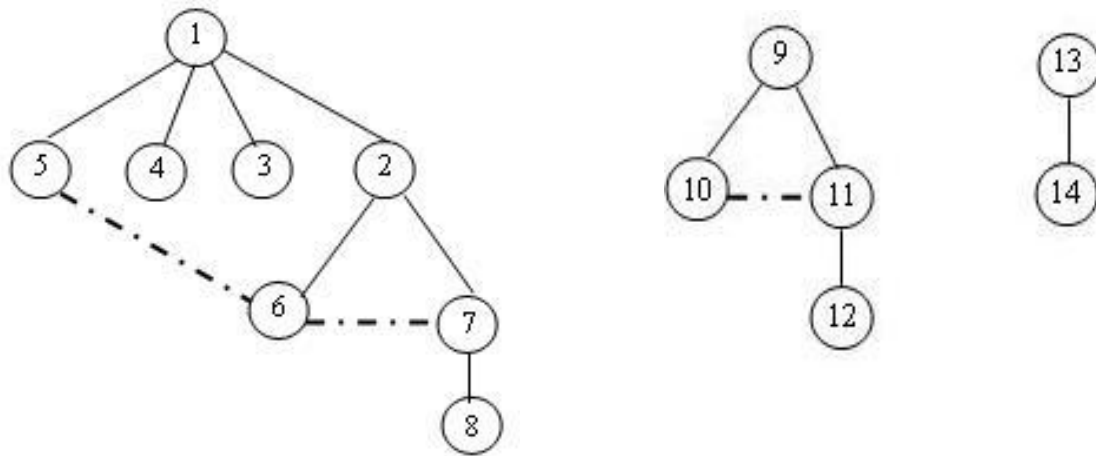
- ✓ Depth-first: 0 1 2 4 3 5 6 8 10 7 9
- ✓ Breadth-first: 0 1 5 2 3 6 4 8 10 7 9

Example: Disjoint Graph – Depth-first

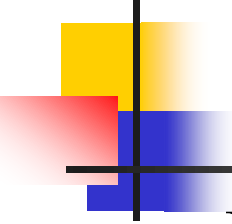


✓ Depth-first traversal: 1 2 6 5 7 8 3 4 9 10 11 12 13 14

- Example: Disjoint Graph – Breadth-first

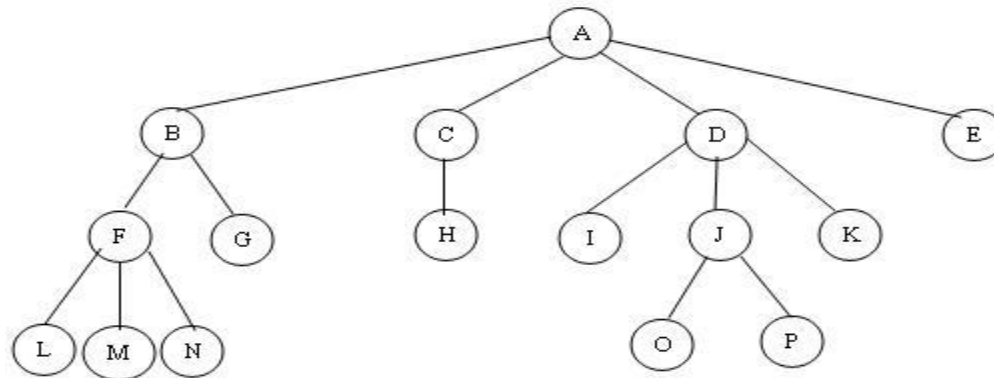


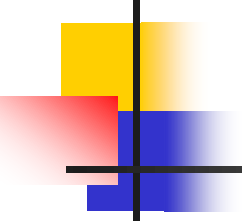
✓ Breadth-first traversal: 1 5 4 3 2 6 7 8 9 10 11 12 13 14



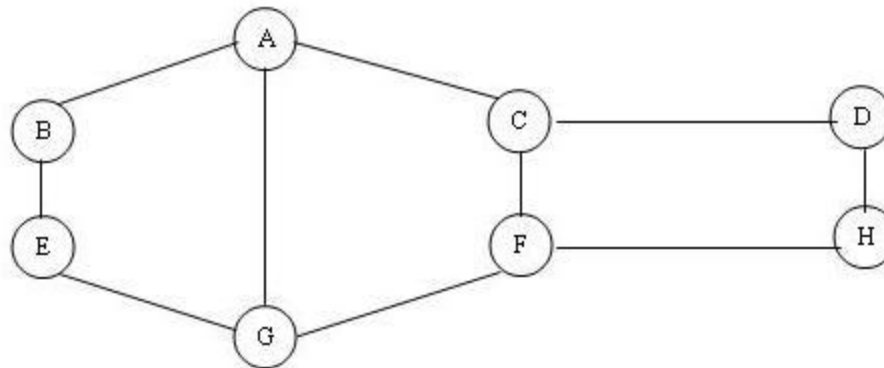
■ Exercise :

1) Give the depth-first and breadth-first traversal for the following tree:





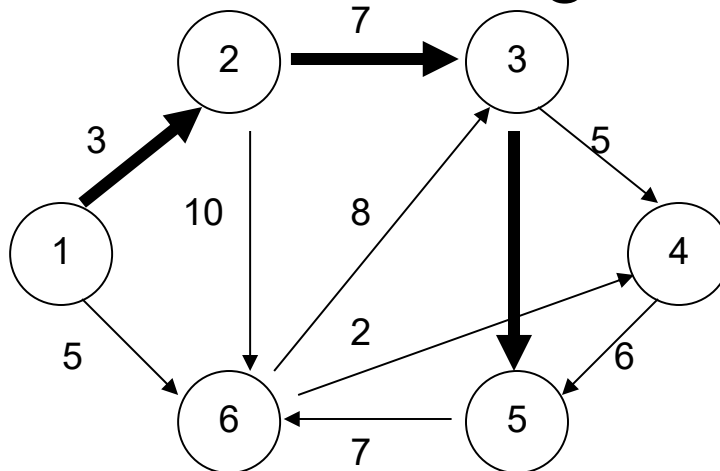
2) Give the breadth-first traversal starting from vertex A.



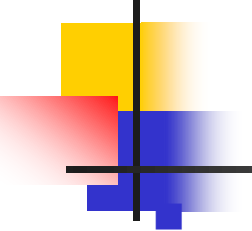
3) Give the adjacency matrix representation of the graph in the above figure.

9.4 Networks

- A network is a graph whose lines are weighted also known as **weighted graph**.
- The meaning of the weights depends on the application such as mileage, time, or cost.



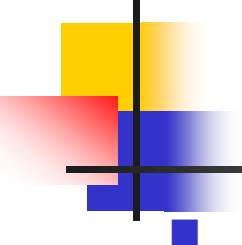
- Two applications of network: the shortest path and minimum spanning tree.



9.4.1 Shortest Path Algorithm

Common application used with graphs is to find the shortest path between two vertices in a network.

- Can use Dijkstra's shortest path algorithm, (developed by Edsger Dijkstra in 1959)
- Dijkstra's Algorithm
 - 1) Pick a vertex, call it W , that is not in S , and for which the distance from 1 to W is a minimum. Add it to S .
 - 2) For every vertex still not in S , see whether the distance from vertex 1 to that vertex can be shortened by going through W . If so, update the corresponding element of Dist .



Pseudocode:

1) $S = \{1\}$

2) $\text{Dist}[2..n] = \text{Edge}[1][2 .. n]$

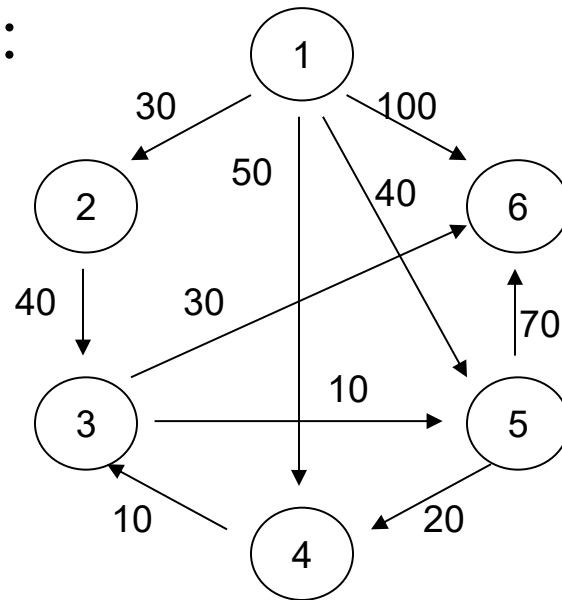
3) for ($i = 1; i \leq n-1$)

 3.1 Choose $W \notin S$ for $\text{Dist}[W]$ is the minimum
 then $S = S + \{W\}$

 3.2 For all vertex $J \notin S$, then

$\text{Dist}[J] = \min(\text{Dist}[j], \text{Dist}[W] + \text{Edge}[W][J])$

■ Example:

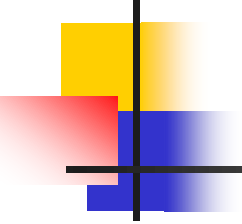


S	W	Dist[2]	Dist[3]	Dist[4]	Dist[5]	Dist[6]
1	-	30	Infiniti	50	40	100
1,2	2	30	70	50	40	100
1,2,5	5	30	70	50	40	100
1,2,5,4	4	30	60	50	40	100
1,2,5,4,3	3	30	60	50	40	90
1,2,5,4,3,6	6	30	60	50	40	90

- Output:

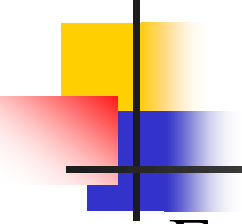
Shortest paths between vertex 1 and all other vertices:

Vertex	Path Length	Predecessor
2	30	1
3	60	4
4	50	1
5	40	1
6	90	3



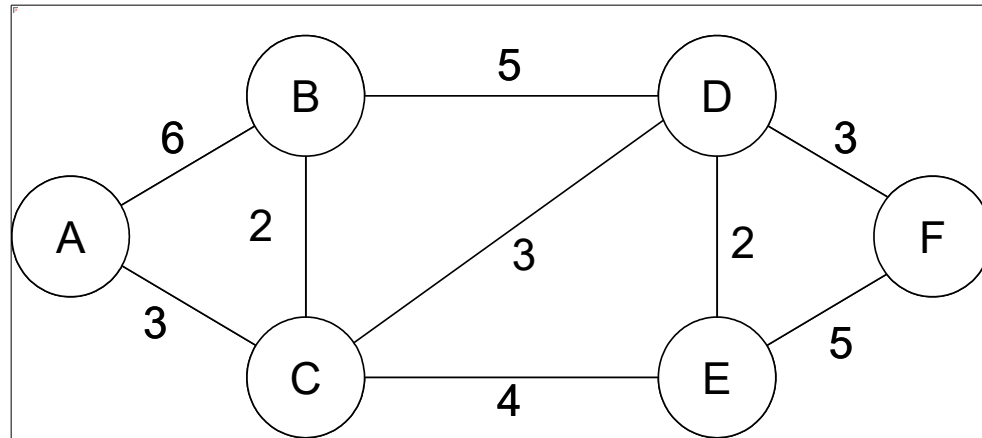
9.4.2 Spanning Tree

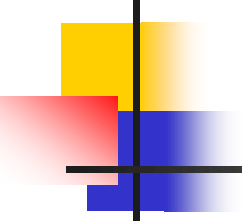
- A spanning tree is a tree that contains all of the vertices in graph.
- A minimum spanning tree is a spanning tree in which the total weight of the lines is guaranteed to be the minimum of all possible trees in the graph.
- Kruskal's algorithm is one of three classical minimum-spanning tree algorithms.



Example :

- To determine the minimum spanning tree shown in Figure 8.



- 
-
- ✓ We can start with any vertex, let start with A.
 - ✓ Then, add the vertex that gives the minimum-weighted edge with A, AC.
 - ✓ The edge AB has a weight of 6, the edge BC has a weight of 2, the edge CD has a weight of 3, and the edge CE has a weight of 4.
 - ✓ Therefore the minimum-weighted edge is BC.
 - ✓ To generalize the process, use the following rule:

From all the vertices in the tree, select the edge with the minimal value to a vertex no currently in the tree and insert it into the tree.

- ✓ Using this rule: add CD (weight 3), DE (weight 2), and DF(weight 3) in turn. The steps graphically shown in Figure 9.

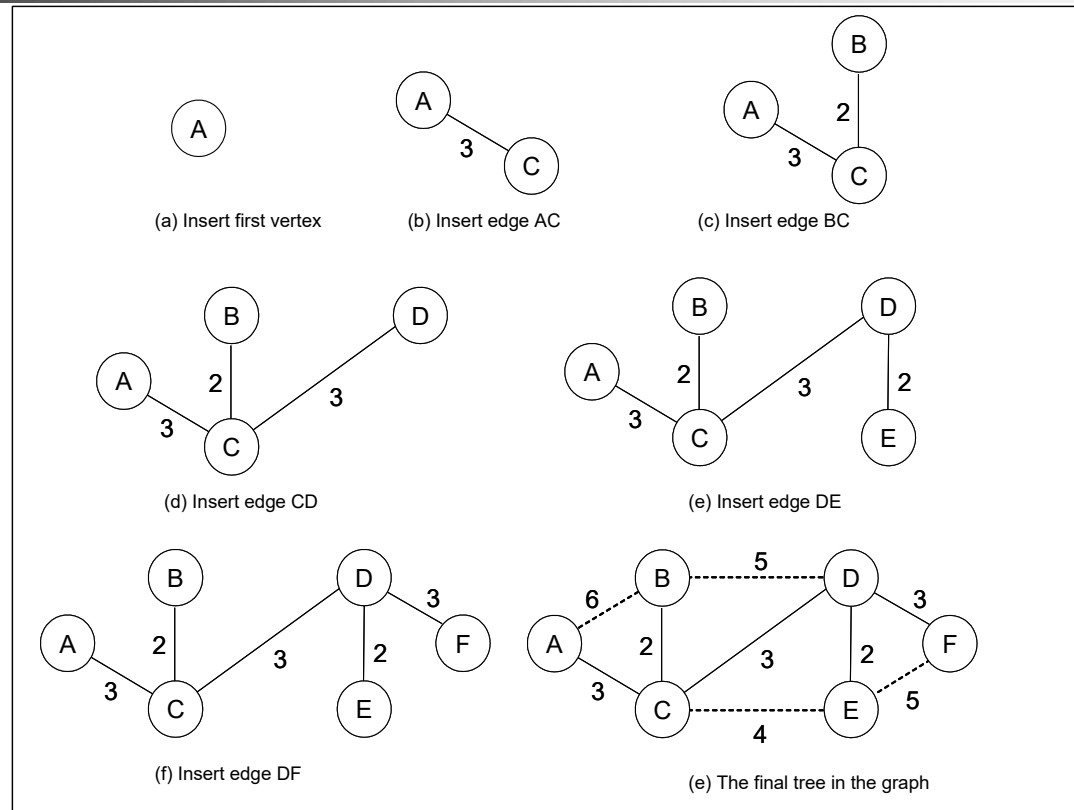
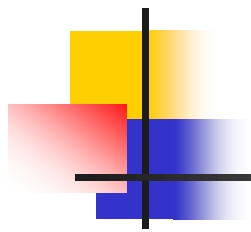
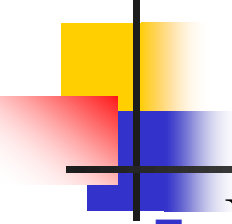


Figure 9 Developing a minimum spanning tree



■ Minimum Spanning Tree Pseudocode

algorithm spanningTree (val graph <metadata>)

Determine the minimum spanning tree of a network.

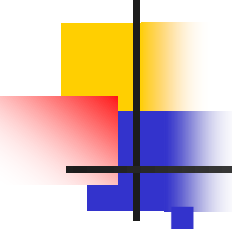
Pre graph contains a network

Post spanning tree determined

1. if (empty graph)
 - 1 return
2. end if
3. vertexPtr = graph.first
4. loop (vertexPtr not null)

■ **Set inTree flags false.**

1. vertexPtr->inTree = false
 2. edgePtr = vertexPtr->edge
 3. loop (edgePtr not null)
 - 1 edgePtr->inTree = false
 - 2 edgePtr = edgePtr->nextEdge
 4. end loop
 5. vertexPtr = vertexPtr->nextVertex
- end loop



Now derive spanning tree.

6 vertexPtr = graph.first

7 vertexPtr->inTree = true

8 treeComplete = false

9 loop (not treeComplete)

1 treeComplete = true

2 chkVertexPtr = vertexPtr

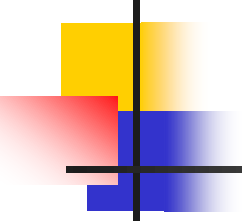
3 minEdge = null

4 minEdgePtr = null

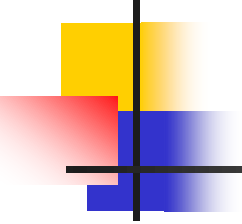
5 loop (chkVertexPtr not null)

walk through graph checking vertices in tree.

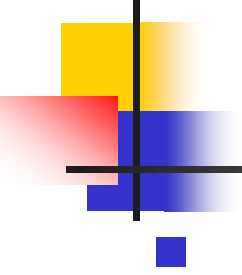
1 if (chkVertexPtr->inTree true
 AND chkVertexPtr->outDegree >0)



```
1 edgePtr = chkVertexPtr->edge
2 loop (edgePtr not null)
    1 if (edgePtr->destination->inTree false)
        1 treeComplete      = false
        2 if (edgePtr->weight < minEdge)
            1 minEdge      = edgeptr->weight
            2 minEdgePtr = edgePtr
        3 end if
    2 end if
    3 edgePtr = edgePtr->nextedge
3 end loop
```



```
2 end if
    chVertexPtr = chkVertexPtr=
    chkVertexPtr->nextVertex
6 end loop
7 if (minEdgePtr not null)
    Found edge to insert into tree.
        1 min DegePtr-> inTree2      = true
        2 minEdgePtr->destination-> = true
8 end if
10 end loop
11 return
end spanning tree
```



Exercise :

3. Based on Diagram 1, find

- a) Breadth-First Traversal starts at vertex A?
- b) Depth-First Traversal starts at vertex A?

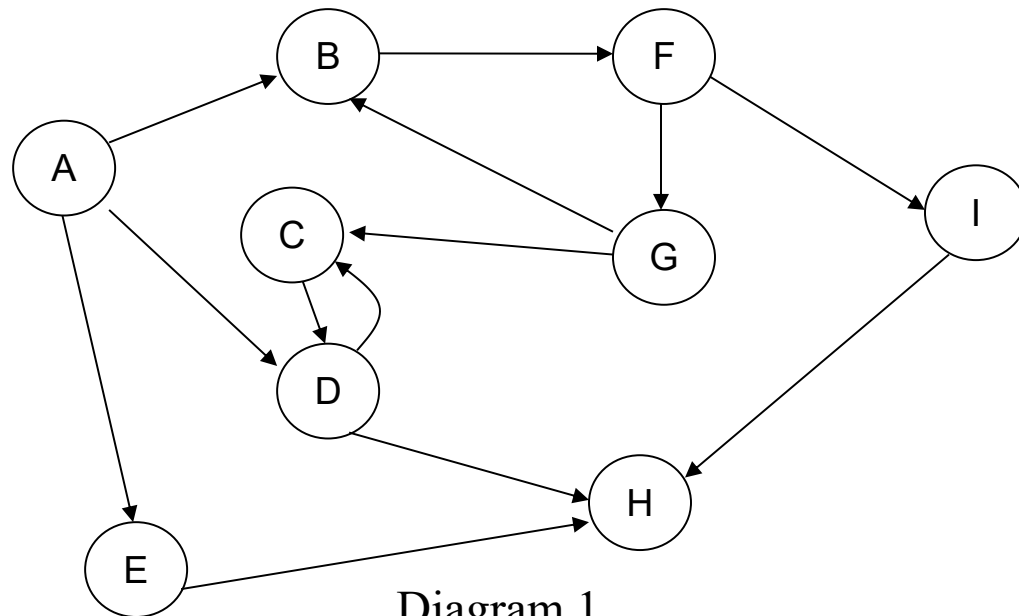


Diagram 1